

BIOL 4559: HW02

Rahul Sunilkumar

2025-09-30

Question 1

Let $\mathbf{A}$ be the adjacency matrix of an undirected network and $\mathbf{1}$ be the column vector with '1' entries. In terms of these quantities write expressions for:

1a, The vector $\mathbf{k}$ whose elements are degrees k_i of the nodes.

Solution Attempt: The degree k_i is the sum of the entries in the i -th row of $\mathbf{A}$. Thus, the vector of degrees is given by the matrix-vector product:

$$\mathbf{k} = \mathbf{A}\mathbf{1}$$

□

1b, The number m of edges in the network.

Solution Attempt: The number of edges is half the sum of all degrees. The sum of all degrees is $\mathbf{1}^T \mathbf{k}$.

$$m = \frac{1}{2} \sum_i k_i = \frac{1}{2} \mathbf{1}^T \mathbf{k} = \frac{1}{2} \mathbf{1}^T \mathbf{A} \mathbf{1}$$

□

1c, The matrix $\mathbf{N}$ whose elements N_{ij} is equal to the number of common neighbors of nodes i and j .

Solution Attempt: The number of common neighbors of nodes i and j is the number of nodes k such that $A_{ik} = 1$ and $A_{jk} = 1$. Since $\mathbf{A}$ is symmetric for an undirected graph, $A_{jk} = A_{kj}$. Thus, $N_{ij} = \sum_k A_{ik}A_{kj}$, which is the definition of the (i, j) -th element of the matrix product $\mathbf{A}\mathbf{A}$.

$$\mathbf{N} = \mathbf{A}^2$$

□

1d, The total number of triangles in the network, where triangle means three nodes, each connected by edges to both of the others.

Solution Attempt: In the previous part we say that the number of common neighbors between any two nodes i and j is given by matrix $N_{ij} = (\mathbf{A}^2)_{ij}$. So, for each edge (i, j) , where there are $(\mathbf{A}^2)_{ij}$ triangles that contain that edge.

To get a total count, we can loop through every edge in the network and then add up the number of triangles it belongs to:

$$\sum_{\text{all edges } (i, j)} (\mathbf{A}^2)_{ij}$$

But, we need to correct for over counting. We count for every edge so in a triangle that's 3 edges, to get the right number of triangles we just need to divide by 3.

$$\frac{1}{3} \sum_{\text{all edges } (i, j)} (\mathbf{A}^2)_{ij}$$

But, this *all edges* thing comes from just multiplying by A_{ij} :

$$\frac{1}{3} \sum_{i, j} A_{ij} (\mathbf{A}^2)_{ij}$$

Which is the same thing but easier to work with. However, we also have now added A_{ij} which double count so we actually need to half it as well giving us a final

$$T = \frac{6}{\sum_{i, j} A_{ij} (\mathbf{A}^2)_{ij}}$$

I've found from online and from my previous math classes that this can be described by 'trace', which makes the coding part easier:

$$\text{Tr}(\mathbf{A}^3) = \sum_i \left(\sum_j A_{ij} (\mathbf{A}^2)_{ij} \right) = \sum_{i, j} A_{ij} (\mathbf{A}^2)_{ij}$$

which gives us a modified final

$$T = \frac{1}{6} \text{Tr}(\mathbf{A}^3)$$

□

1e, Calculations from Adjacency Matrix

From the dolphin social network (see Canvas), create an unweighted adjacency matrix by first assigning '1' if the matrix entry is 5 or larger and zero otherwise and then setting the matrix diagonal equal to 0. Using this post-processed adjacency matrix, the ones vector, and the expressions you derived above to calculate.

Solution Attempt: Before we start we should import the following,

```
import polars as pl
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
```

1e (i), Degree of Each Node.

First, we download our data and put it in a matrix. I've found this `to_numpy()` function that makes this into numpy arrays that should help us edit it easier.

```
df = pl.read_csv('../data/dolphin.csv', has_header = False)
matrix = df.to_numpy() # Nice function I found online.
matrix
```

```
array([[12, 12,  8,  7,  7,  4,  7,  4,  0,  0,  0,  0,  0],
       [12, 12,  8,  7,  7,  4,  7,  4,  0,  0,  0,  0,  0],
       [ 8,  8, 24, 23, 18,  3,  5,  3,  0,  0,  0,  0,  0],
       [ 7,  7, 23, 26, 19,  2,  4,  2,  0,  0,  0,  0,  0],
       [ 7,  7, 18, 19, 20,  3,  5,  3,  0,  0,  0,  0,  0],
       [ 4,  4,  3,  2,  3, 21, 20, 21,  0,  0,  0,  0,  0],
       [ 7,  7,  5,  4,  5, 20, 23, 20,  0,  0,  0,  0,  0],
       [ 4,  4,  3,  2,  3, 21, 20, 21,  0,  0,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0, 31, 26,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0, 26, 28,  0,  0,  0],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 35, 31, 24],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 31, 31, 22],
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 24, 22, 25]])
```

And from the directions, we accomplish the zeroes and ones matrix with the following.

```
A = (matrix >= 5)*1 # `*1` is indicator function.
np.fill_diagonal(A,0) # Filling diagonal as asked.
A
```

```
array([[0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0],
       [1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0],
       [1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0],
       [1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
       [1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0],
       [1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0]])
```

This gives us our answer,

```
num_nodes = A.shape[0]
ones = np.ones((num_nodes, 1)) # Getting  $\mathbf{1}$  as asked.

# Degree of each node
# Formula:  $k = A @ 1$ 
k = A @ ones
print(f'Degree of each node (k): {k.astype(int).flatten()}')
```

Degree of each node (k): [5 5 5 4 5 2 6 2 1 1 2 2 2]

1e (ii), Number of Edges.

```
m_matrix = 0.5 * ones.T @ A @ ones # class formula.
num_edges = int(m_matrix.item()) # `.item()` is a nice func to get scalar val.
print(f'Total number of edges (m): {num_edges}')
```

Total number of edges (m): 21

1e (iii), Number of common neighbors for nodes 0 and 3.

This follows straight from our discussion earlier.

```
# Number of common neighbors for nodes 0 and 3
# Formula:  $N = A^2$ 
N = A @ A
common_neighbors_0_3 = N[0,3]
print(f'Number of common neighbors for nodes 0 and 3: {common_neighbors_0_3}')
```

Number of common neighbors for nodes 0 and 3: 3

1e (iv), The total number of triangles.

We've discussed how we've arrived at the trace function in the earlier parts of this question. This was on purpose because the `.trace` function is readily available with `numpy`.

```
# Total number of triangles
# Formula:  $T = (1/6) * \text{Tr}(A^3)$ 
A_cubed = np.linalg.matrix_power(A, 3)
trace_A3 = np.trace(A_cubed) # From documentation.
num_triangles = int(trace_A3 / 6)
print(f'Total number of triangles: {num_triangles}')
```

Total number of triangles: 18

□

Question 2

For this question you will be using `BTSFlights` data on Canvas. Use the snippet & data from Canvas to do the following.

Solution Attempt: Get our data first with snippet

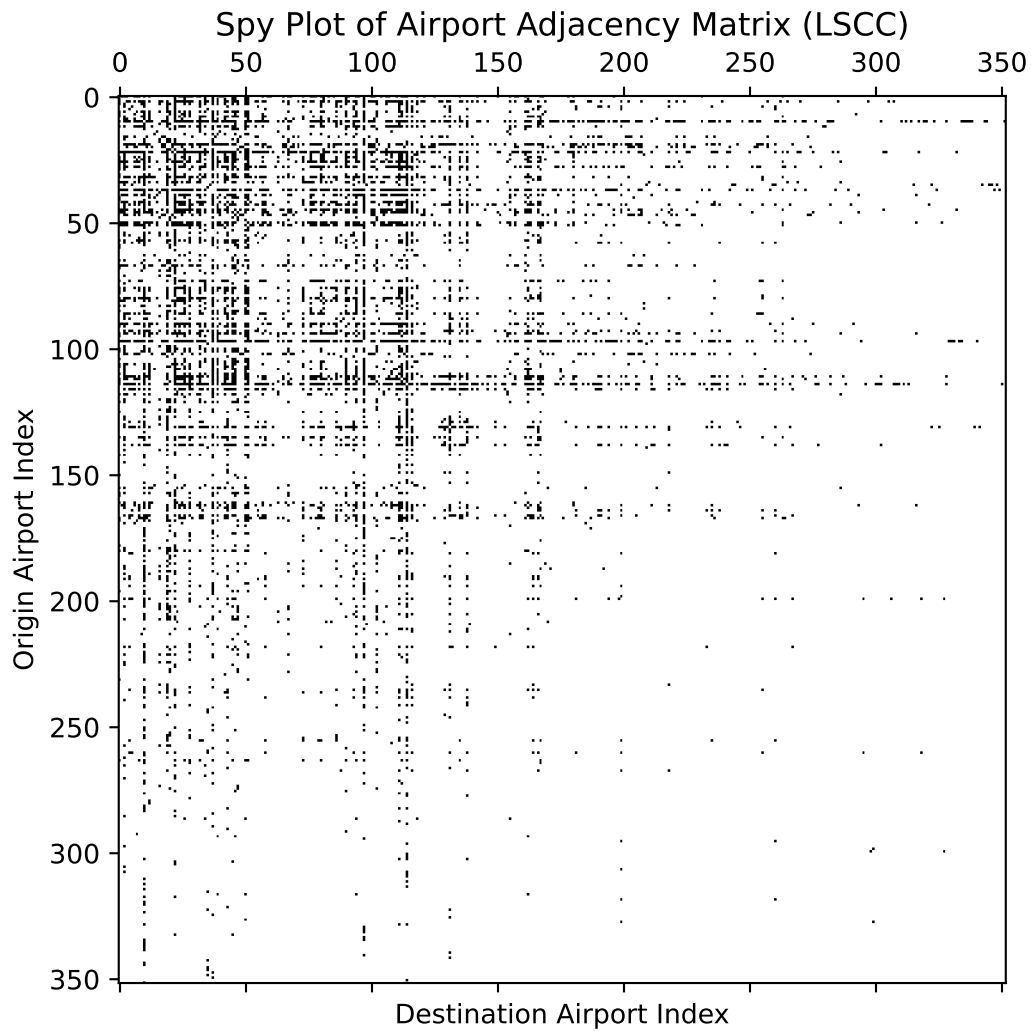
```
df = pl.read_csv('../data/bts2025.csv')
df = df.filter(pl.col('PASSENGERS') > 1000)

G = nx.DiGraph(df[['ORIGIN', 'DEST']].rows())
lsc = max(nx.strongly_connected_components(G), key = len)
G = G.subgraph(lsc).copy()

# Variables that you may need.
nodedict = {node : i for i, node in enumerate(G.nodes)}
reverse_nodedict = {i : node for i, node in enumerate(G.nodes)}
A = nx.to_numpy_array(G, nodelist = nodedict.keys(), dtype = int)
```

2a, Draw adjacency matrix with a spy plot.

```
plt.figure(figsize = (6, 6))
plt.spy(A) # Simply pulled from docs.
plt.title('Spy Plot of Airport Adjacency Matrix (LSCC)')
plt.xlabel('Destination Airport Index')
plt.ylabel('Origin Airport Index')
plt.show()
```



2b, Using the adjacency matrix, calculate the diameter of this network.

This one is fairly straightforward with `networkx` functions.

```
diameter = nx.diameter(G)
print(f'The diameter of the airport network is: {diameter}')
```

The diameter of the airport network is: 6

2c, and the number of airports that you can reach from Charlottesville (with airport code CHO) by taking (i) one, (ii) two, (iii) three, and (iv) four flights.

This is a little more involved but we can cycle through and count the occurrences of each distance.

```
# Gets shortest path lengths from CHO to all other nodes!
path_lengths = nx.single_source_shortest_path_length(G, 'CHO')

# Get a list of all the distances.
distances = list(path_lengths.values())

# Loop & count the occurrences of each distance.
for i in range(1, 5):
    count = distances.count(i)
    print(f"Airports reachable in exactly {i} flight(s): {count}")
```

```
Airports reachable in exactly 1 flight(s): 5
Airports reachable in exactly 2 flight(s): 192
Airports reachable in exactly 3 flight(s): 150
Airports reachable in exactly 4 flight(s): 3
```

2d, What are the top 5 airports in the US by degree and what are their degrees?

The difficult part here is getting that `degree_list` which we utilize the code snippet given to loop through the shape and sort it. This makes our lives a lot easier and then we loop through from the greatest to the fifth greatest.

```
# For a directed graph, we'll consider total degree (in + out).
# In-degree = sum of columns, Out-degree = sum of rows.
in_degrees = A.sum(axis = 0)
out_degrees = A.sum(axis = 1)
total_degrees = in_degrees + out_degrees

degree_list = [] # empty list.

# A.shape[0] gives us the number of rows in our matrix A,
# which is the number of airports.
num_airports = A.shape[0]
```



```

for i in range(num_airports):
    degree = total_degrees[i]           # Get deg using index.
    airport_name = reverse_nodedict[i]  # Get name using index.
    airport_info = (degree, airport_name) # Create tuple.
    degree_list.append(airport_info)     # Add to list.

degree_list.sort(key = lambda x : x[0], reverse = True) # `.sort` ranks it.
# `lambda` is just a helper shortcut to get the first
# item in the tuple (which is the degree).

print("Top 5 airports by total degree:")
for degree, airport in degree_list[:5]: # After ranking, we go from 1st to 5th.
    print(f"{airport}: {degree} degree")

```

Top 5 airports by total degree:

DFW: 354 degree

DEN: 326 degree

ATL: 306 degree

ORD: 297 degree

CLT: 266 degree

2e, Plot a histogram of the network's degree distribution, with enough resolution that you can see interesting behavior. Why do you think the airport network has this kind of degree distribution.

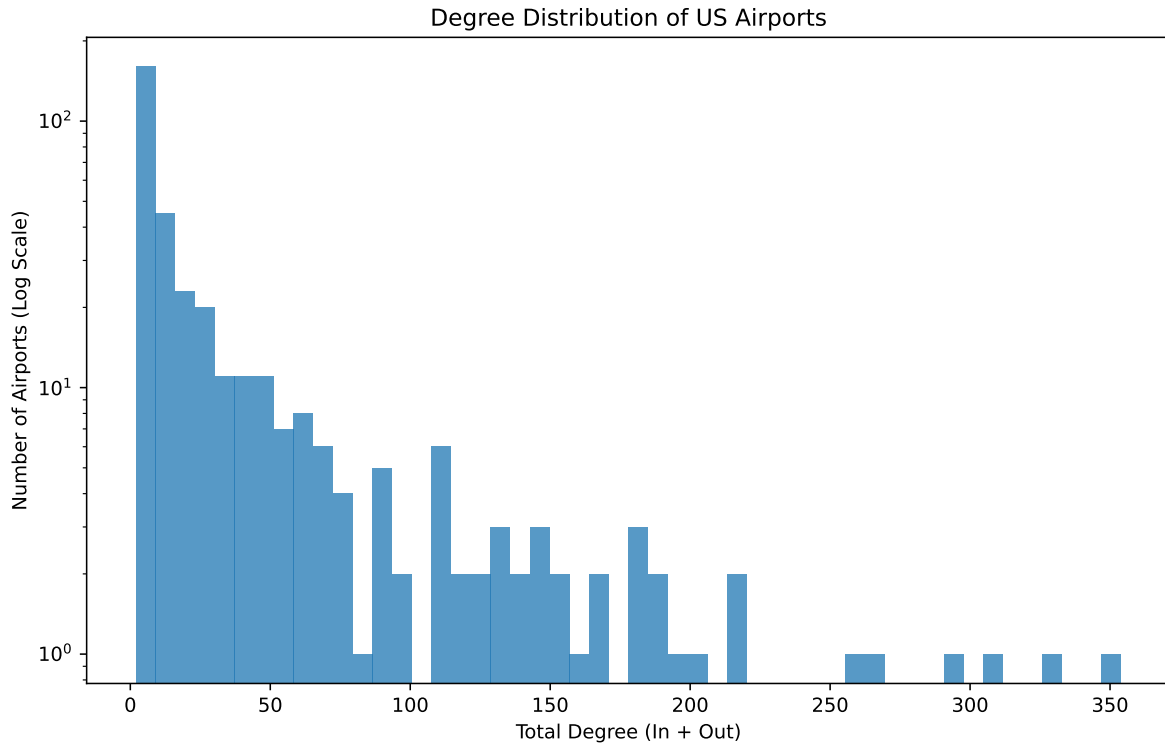
```

plt.figure(figsize = (10, 6))
plt.hist(total_degrees, bins = 50, alpha = 0.75) # Hist

plt.yscale('log') # log scale.

# Details
plt.title("Degree Distribution of US Airports")
plt.xlabel("Total Degree (In + Out)")
plt.ylabel("Number of Airports (Log Scale)")
plt.show()

```



The airport network exhibits a ‘long-tailed’ or ‘scale-free’ (which we just discussed in class earlier) distribution. A vast majority of the airports have a very low degree (small, regional). A small number of airports act as major hubs and have a high degree. This structure is efficient for connecting many locations with a small number of total routes.

□